

Day 4



amazon



Infosys

tcs



90 Days SDET
Interview Prep

Please make sure to take the printout of this particular document for studies.

90 days SDET Interview Prep

Day 4: Abstract Classes and Interfaces

1. What is an Abstract Class?

An **abstract class** in Java is a class that is **incomplete by design** and is meant to be **extended**, not instantiated.

This means that Abstract Classes are designed to be Parent Classes.

Key Points

- An abstract class **may or may not** contain abstract methods.
- If a class has **at least one abstract method**, the class **must be marked abstract**.
- You **cannot create an object** of an abstract class using the new keyword.
- Abstract classes **can have**:
 - ✓ Constructors
 - ✓ Instance variables
 - ✓ Concrete (implemented) methods
 - ✓ Abstract methods

2. Can an Abstract Class Exist Without Abstract Methods?

Yes. You can have classes which are abstract classes that don't have abstract methods! When you mark the class as Abstract you are restricting OBJECT creation.

Why do this?

- To **prevent direct object creation**
- To act as a **base template**
- To enforce **inheritance:based design**

This is very common in **automation frameworks**.

3. What is an Interface?

An **interface** is a **100% contract-based structure** (pre:Java 8) that forces method overriding and defines **what a class must do**, not how.

Key Properties

- Methods are **public & abstract by default**
- Variables are **public, static, and final [CONSTANT]**
- No object creation
- No constructors

Since Java 8+, interfaces can have:

- ✓ Default methods (with implementation)
- ✓ Static methods (with implementation)
- ✓ Private methods (since Java 9)

4. When to Use Abstract Class vs Interface in Test Automation

When to Use Abstract Class?

Use Abstract Class when:

- ***You want to share state and behavior*** . Abstract classes can have instance variables (state) along with method implementations that can be inherited by subclasses
- You have a base test class : When you need a foundation class that provides common setup, teardown, and utility methods for all your test classes
- You want to provide common reusable logic : When multiple related classes share the same implementation code that shouldn't be duplicated

Automation Examples:

- **BaseTest** : Contains common setup methods like browser initialization, configuration loading, before/after hooks, and reporting setup that all test classes inherit
- **BasePage** : Provides common page operations like waits, element interactions, and navigation methods that all page object classes can reuse

When to Use Interface?

Use Interface when:

- **You want to define a contract** : Interfaces specify what methods a class must implement without dictating how, ensuring consistency across implementations
- **Multiple unrelated classes need same behavior** : When different classes that don't share a common ancestor need to implement the same set of methods
- **You need multiple inheritance** : Java doesn't support multiple class inheritance, but a class can implement multiple interfaces, allowing you to combine different contracts

Automation Examples:

- **Listener contracts** : TestNG listeners like ITestListener, ISuiteListener define methods that must be implemented for custom test execution hooks.
- **RequestSpecification** : RestAssured interface that defines the contract for building HTTP request specifications
- **Response** : RestAssured interface that defines methods for handling and validating HTTP responses.
- **Filters(Function Interface)** : RestAssured filter interface that allows custom request/response logging and modification logic

5. What is a Functional Interface?

- A Functional Interface is an interface that contains exactly **ONE abstract method**.
It's also called a SAM (Single Abstract Method) interface.

Key Characteristics:

- ✓ Exactly 1 abstract method (mandatory)
- ✓ Can have multiple default methods
- ✓ Can have multiple static methods
- ✓ Can have methods inherited from Object class (equals, hashCode, toString)
- ✓ Marked with @FunctionalInterface annotation (optional but recommended)

Functional interfaces are the foundation of Lambda Expressions and Functional Programming in Java 8+.

BONUS Questions:

6. Can an interface extend another interface?

- Yes, and it can extend multiple interfaces. Parent Interface is called SUPER Interface
- Example: SearchContext

7. Can an abstract class implement an interface?

- Yes, and it doesn't need to implement all methods (they remain abstract).

8. What happens if a class implements two interfaces with the same default method?

- Compile-time error. The class must override the method to resolve the conflict.

9. Marker Interface vs Functional Interface?

- Marker interface: 0 methods (e.g. Serializable)
- Functional interface: exactly 1 abstract method

Tech with
Jatin



**JAVA
SDET
MASTERCLASS**

Join Now